



From engineering models to digital twins: Generating AAS from SysML v2 models

Enxhi Ferko^a, Luca Berardinelli^{b,*}, Alessio Bucaioni^a, Moris Behnam^a,
Manuel Wimmer^b

^a Mälardalen University, Västerås, Sweden

^b Johannes Kepler University Linz, Linz, Austria

ARTICLE INFO

Editor: Professor Laurence Duchien

Keywords:

Digital twin
SysML
Asset administration shell
Interoperability
Model-driven engineering
Model transformation

ABSTRACT

Context: Digital twins serve as virtual representations of systems, enabling capabilities such as intelligent monitoring, real-time control, decision-making, and predictive analytics. The Asset Administration Shell (AAS) is the pivotal Industry 4.0 standard for digital twin engineering. In parallel, the Systems Modeling Language (SysML) has emerged as a modeling standard for systems engineering, providing a formalized and semantically rich approach to system modeling. SysML v2 is its recent evolution. With its growing adoption, multiple models are expected to be widely available, each capturing different facets of the modeled system by leveraging diverse engineering capabilities offered by various tool ecosystems. **Problem:** Instead of manually re-creating models for digital twinning, existing system models should be leveraged to relieve repetitive modeling tasks. While SysML v2 and AAS are prominent standards in DT engineering, they lack direct integration, necessitating a dedicated approach for their seamless interoperability. **Purpose:** This paper presents a practical investigation into the conceptual alignment between the SysML v2 and AAS specifications, with a focus on their structural and behavioral modeling aspects. It proposes an implementable approach for mapping SysML v2 to AAS, enabling the automated generation of AAS models from SysML v2 models. **Method:** To realize this approach, we employ model-driven engineering techniques leveraging the Eclipse Modeling Framework (EMF) and model transformations based on the Query View Transformation (QVT) language. The proposed model transformation incorporates query mechanisms for extracting structured elements, preserving information and structural integrity, and ensuring static semantic consistency at design-time and seamless integration between the two investigated standards. We develop and validate the model transformation following an iterative test-driven development approach using an existing set of 24 SysML v2 examples, sourced from the official SysML v2 repository. **Result:** We deliver a QVT-based, EMF-compliant transformation that automatically generates AAS submodel templates from SysML v2 models, preserving structural hierarchies and behavioral semantics via dedicated AAS concepts and their extension. Through an iterative, test-driven development process, we validate metamodel conformance, information preservation, and structural integrity. The current mapping addresses design-time concepts, and the implementation supports forward transformation. All conceptual mappings, QVT scripts, and example artifacts are publicly available in a dedicated repository.

1. Introduction

Digital twins (DTs) became a foundational technology for enhancing the understanding, design, operation, and management of complex systems (Zhang et al., 2025; Guzina et al., 2022). The functional suitability of a DT depends on its ability to accurately capture, model, and exchange data with its physical counterpart. Consequently, ensuring seamless interoperability between different components of a DT and across multiple DTs is crucial (Ferko et al., 2022). Despite attempts to develop in-

teroperability frameworks, DT engineering remains largely fragmented and lacking robust engineering approaches, complicating bidirectional data exchange between the engineering models¹ of physical systems and their digital twins (Schmidt et al., 2023; Platenius-Mohr et al., 2020; Zhang et al., 2025).

¹ With this term, we refer to various types of models developed during the design and planning phases

* Corresponding author.

E-mail address: luca.berardinelli@jku.at; luca.berardinelli.jku@gmail.com (L. Berardinelli).

<https://doi.org/10.1016/j.jss.2025.112688>

Received 2 March 2025; Received in revised form 12 October 2025; Accepted 27 October 2025

Available online 6 November 2025

0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

A promising solution to address these interoperability challenges is the reuse of existing engineering models for deriving DT models aligned with established DT standards, such as Asset Administration Shells (AASs), rather than manually re-creating DT models from scratch (Pereira Pessoa et al., 2023; Zhang et al., 2025). In manufacturing and industrial automation, numerous models are already generated during the engineering phases of production machine design, factory planning, and process modeling (Platenius-Mohr et al., 2019; Zhang et al., 2025). Leveraging these pre-existing models could significantly accelerate DT creation, enhance standardization, and improve the accuracy of digital representations. However, integrating engineering models into DT frameworks requires systematic transformation methods that can extract, structure, and adapt this information for use in DT environments (Luder et al., 2020; Miny et al., 2023a). Without such methods, valuable models remain unutilized, and tedious, repetitive work is required.

In this paper, we employ Model-Driven Engineering (MDE) techniques, adopting the Eclipse Modeling Framework (EMF) as technical space (Bézivin, 2006) to establish interoperability between the Systems Modeling Language version 2 (SysML v2), a prominent systems engineering language (Jansen et al., 2022), and the AAS, a standard for DT engineering (IDTA-Workstream "Specification of AAS", 2025).

The main contribution of this paper is two-fold. First, we analyze the SysML v2 and AAS language specifications to examine their commonalities and differences and identify conceptual mapping possibilities. By this, we extend the mappings defined in our previous paper (Fferko et al., 2024). Second, building on the conceptual mappings, we developed a model transformation that leverages Query/View/Transformation Operational Mapping Language (QVTo)², an imperative rule-based model transformation language. Our implementation leverages i) a query mechanism for extracting structural and behavioral elements from SysML v2 models and generating AAS-compliant models, thereby ensuring the preservation of structural and behavioral semantics in the transformation process from SysML v2 to AAS; ii) the AAS extension mechanisms to enrich the generated AAS models with SysML v2-specific annotations, facilitating potential reverse transformations back to SysML v2.

We developed and validated the transformation following an iterative test-driven development approach (Troya et al., 2023), on a set of 24 SysML v2 case studies, sourced from the SysML v2 repository (Object Management Group, 2022) obtaining AAS models conforming to the proposed Ecore-based AAS metamodel implementation. Such transformations enable the automated generation of AAS models from SysML v2 models, ensuring semantic alignment and traceability. We evaluate the transformation with respect to information and structural preservation, while leaving performance evaluation and full language coverage to future work. We provide a publicly accessible repository (Fferko et al., 2025), which contains all developed and descriptive artifacts.

It should be noted that while part of the first contribution is based on our previous work (Fferko et al., 2024), the second contribution is entirely novel. Furthermore, compared to the mapping discussed in our precursor work, this paper extends it by incorporating additional structural and behavioral elements related to *State machines* in SysML v2 and investigating mapping patterns and their rationale. Finally, we discuss several aspects of the proposed mapping, including current limitations. Taken together, our contributions aim to advance the state of the practice in model-driven digital twin development through concrete transformation techniques grounded in standard specifications.

The remainder of this paper is structured as follows. Section 2 gives an overview of fundamental concepts used in this work and elaborates on related work. Section 3 details the research methodology. Section 4 presents the conceptual metamodels of KerML, SysML v2, and AAS considered in the context of this work. Section 5 elaborates on map-

pings between SysML v2 and AAS metamodels. Section 6 presents the model transformation and its evaluation. We discuss our experiences and lessons learned in Section 7. Finally, we present the conclusions and future work in Section 8.

2. Background and related work

This section provides an overview of the fundamental concepts underlying our research and examines the relevant literature.

2.1. Digital twin and asset administration shell

Digital twin (DT). A DT is a virtual representation of an *actual system* (AS) that is continuously updated with real-time data throughout its life cycle and, at the same time, can interact with and influence the system, leveraging a bidirectional data exchange. The term DT evolved from its first appearance (Grieves, 2014) towards various definitions employed across industry and academia in different domains (Jones et al., 2020; Dalibor et al., 2022). Among them, the ISO 23247 standard defines “a digital twin in manufacturing as a fit-for-purpose digital representation of an observable manufacturing element with synchronization between the element and its digital representation[...]” (International Organization for Standardization, 2021). Kritzinger et al. introduce further concepts based on different communication capabilities, such as the Digital Model (DM) as a digital representation without data exchange with the AS, and Digital Shadow (DS) as purposefully abstracted and aggregated data structures from the AS with one-way automated data flow from the AS to DT (Kritzinger et al., 2018). In Eramo et al. (2022), Eramo et al. propose a conceptual modeling framework for DT based on the models, data, and transformations (Combemale et al., 2021) where heterogeneous models (engineering, scientific, AI ones), playing different roles (descriptive, prescriptive, predictive), are integrated within the DM to accomplish specific goals in particular contexts.

Asset administration shell (AAS). AAS is a prominent standard in DT engineering promoted by the International Digital Twin Association (IDTA) (Industrial Digital Twin Association, 2025) and Platform Industry 4.0 (Plattform Industrie 4.0, 2025). It is the dominant standard to enable interoperability and standardized communication among diverse manufacturing entities (Fferko et al., 2023), representing and making accessible the asset along its complete life cycle (International Electrotechnical Commission, 2020).

With the AAS standard, the AS is thus described as a collection of physical and non-physical assets. Assets can include production resources, workpieces, software, and even licenses (Frysak et al., 2018). An AAS model must conform to the AAS metamodel by the IDTA, which is documented via the Unified Modeling Language (UML) without being tied to a specific implementation technology (Industrial Digital Twin Association, 2025).

An AAS model is typically structured as a hierarchy of submodels, each representing distinct components or aspects of the asset (e.g., a motor, drive train, or energy efficiency for an e-vehicle). This modular approach facilitates the systematic integration of various types of data and functionalities into a coherent DT framework (e.g., Jacoby et al., 2022). An AAS model should encapsulate all relevant information about an asset, including properties, operational parameters, and historical data to support the realization of a DT.

An AAS model can be classified into three types based on their capabilities and requirements (Zhang et al., 2025): Type 1 AASs store static data, thus dealing mostly with the DM. Type 2 AASs connect these models to live data from the asset, providing an API for interacting with other components, adding live data to the models to Type 1 AAS, and thus, creating the DS. Type 3 AAS or reactive AAS provides a comprehensive solution for the engineering and management of DT according to the RAMI 4.0 layers (Jacoby et al., 2022; DIN SPEC 91345:2016-04, 2016). Section 4.2 dives into details of the AAS metamodel.

² <https://wiki.eclipse.org/QVTo/>

2.2. Model-driven engineering and SysML v2

MDE is a software and system engineering approach that mitigates platform complexity and efficiently represents domain concepts through the systematic use of models and modeling languages—as defined by metamodels, and model transformations (Schmidt, 2006; Burgueño et al., 2019; Brambilla et al., 2017). Models are fundamental building blocks in the software development process and serve as purposeful representations of systems, created in conformance with general-purpose or domain-specific modeling languages whose concepts and their relationships are defined by metamodels (Bézivin, 2005).

The Object Management Group (OMG) serves as a leading standards consortium that defines and maintains critical specifications, including the Meta Object Facility (MOF), Unified Modeling Language (UML), and Systems Modeling Language (SysML), to advance MDE by enabling consistent and interoperable software and system modeling across diverse domains.

Recently, the OMG has developed and launched two complementary standards: the Kernel Modeling Language (KerML) and SysML v2. KerML is a foundational modeling language that serves as the semantic foundation for systems modeling languages, particularly SysML v2. KerML provides a minimal, mathematically precise kernel of fundamental modeling primitives, including elements, relationships, and namespaces, while introducing essential classification mechanisms based on Tarsky's model theory (Almeida et al., 2025). The combination of MOF and KerML represents a fundamental paradigm shift in modeling language design by the OMG, transitioning from purely structural language definition, as traditionally supported by MOF, to semantically rich specification capabilities enabled through KerML.

SysML v2 (Object Management Group, 2024b, 2022) is the latest evolution of the SysML standard. SysML v1 is one of the most widely used standard modeling languages for complex systems, conceived to overcome the limitations of the general-purpose UML in capturing requirements, structures, behaviors, and constraints across multidisciplinary systems. SysML v1 has been conceived as a UML heavyweight extension since its first official release by OMG in 2007. Over the past two decades, SysML v1 achieved widespread adoption (Wolny et al., 2020). SysML v2 is now decoupled from UML. Its new metamodel is now designed as a specialization of KerML to enhance precision, expressiveness, and life-cycle integration via a standardized API for better support of MDE approaches (Jansen et al., 2022; Almeida et al., 2025; Bajaj et al., 2022). SysML v2 supports requirements, structural, and behavioral modeling using textual notation. A graphical notation support is in progress. The reference implementation supports visualization via PlantUML while free (e.g., Eclipse SysON³) and commercial solutions are offering both textual and graphical modeling capabilities. Section 4 dives into details of the KerML and SysML v2 metamodels.

Model transformations are specialized software programs designed to automate model manipulation (Czarnecki and Helsen, 2006). They can be used for various purposes, including the generation or synchronization of models at different levels of abstraction, the creation of various views on a system, and the automation of model evolution tasks. In particular, model transformations can facilitate automated translations between different modeling languages by mapping the concepts from a source language model to those of a target language model (Czarnecki and Helsen, 2006), realizing different transformation intents and properties (Lúcio et al., 2016).

2.3. Related work

Our work focuses primarily on bridging the gap between SysML v2 and AAS, an area that, to the best of our knowledge, has not been explored in prior research. In this section, we review relevant research in

the field of model transformation, focusing on three key areas: model transformations from SysML v1 to other formalisms, SysML v2-specific transformations, and transformations from modeling languages to AAS. Transformations from SysML v1 to other formalisms. Pessoa et al. proposed an eight-step system-based modeling approach, which integrates various models to achieve a realistic representation of the system (Pessoa et al., 2023). Their approach includes guidelines to streamline SysML usage in Eclipse Papyrus and a Python parser to convert SysML models into simulation models. An application example demonstrated its feasibility, showing that the proposed methodology and tooling help reduce the model-driven learning curve and improve SysML usability in DT design. This work supports the core idea behind our research and shares commonalities with our approach, particularly in leveraging SysML. However, unlike our method, it relies on SysML v1 and lacks a systematic transformation to AAS.

A recent study proposed integrating AAS with MDE to meet the stringent requirements of Industry 4.0 environments (Molina and Treichel, 2023). This integration enables real-time data collection and system model updates, extending model longevity and utility. The authors developed a MATLAB/Simulink library using BaSyx APIs, facilitating communication with sub-models stored in a BaSyx AAS server. Their approach was validated through a hydraulic press simulation, demonstrating its effectiveness in condition monitoring. Like Pessoa et al., this study relies on SysML v1, limiting its applicability to more recent advances in system modeling. A systematic mapping study by Wolny et al. provides an extensive review of SysML transformations over thirteen years. Their study introduces a formalism transformation graph, which summarizes transformations to and from SysML from 579 included studies (Wolny et al., 2020).

SysML v2-specific transformations. The OMG has published standardization efforts for SysML transformations, including specifications for SysML v1 to v2 transformation (Object Management Group, 2023) and implementation in QVTo (Object Management Group, 2022). This standardization provides a foundation for implementing model transformations involving SysML v2, establishing formal approaches for migrating and transforming models between SysML versions. Heermann et al. proposed a method for generating DT data from SysML v2 models, focusing on real-time DevOps integration and runtime monitoring (Heermann et al., 2024). Their approach leverages SysML v2 textual models to define DT structures and communication interfaces. Our approach differs from theirs as we focus on developing a model transformation to generate DT models compliant with an EMF-based implementation of the AAS standard, ensuring alignment of structural and behavioral information. While both approaches leverage SysML v2 for DT applications, our work is complementary, as real-time DT models could benefit from AAS-based standardization.

Transformations from modeling languages to AAS. Miny et al. reviewed various languages for describing automation devices, comparing selected languages and discussing integration methods (Miny et al., 2023b). They explored combining Web of Things (WoT), Thing Description (TD), and AAS metamodels based on Pakala et al.'s mapping rules (Pakala et al., 2021). Unlike their conceptual approach, we propose a systematic method for model transformations. Lüder et al. identified AAS as the standard for Industry 4.0 components and addressed data integration from heterogeneous sources using AutomationML (Lüder et al., 2020). While their goal aligns with ours, namely representing data in AAS, they do not address automated translation mechanisms. Several studies examined OPC UA and AAS integration. Cavalieri et al. detailed OPC UA's information model and AAS metamodel, proposing mappings and a tool for conversion (Cavalieri and Salafia, 2020). Fuchs et al. focused on mapping rules between these languages (Fuchs et al., 2019). Unlike these approaches, we employ MDE techniques to translate existing models into AAS. Schmidt et al. proposed a JSON-based mapping from DTDL to AAS, demonstrating that each DTDL element can be mapped to AAS (Schmidt et al., 2023). Platenius-Mohr et al.

³ <https://mbse-syson.org/>

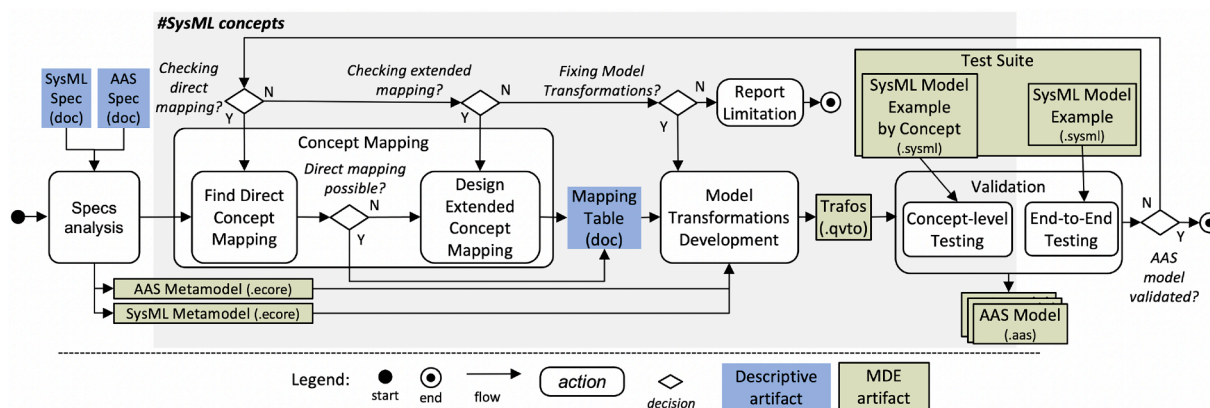


Fig. 1. Overview on SysML v2 to AAS structured integration approach.

defined interoperability requirements for DTs, proposing a model transformation-based solution validated in an industrial setting (Platenius-Mohr et al., 2020). Our approach complements theirs by also leveraging model transformations for on-demand interoperability.

Synopsis. Although previous research has explored SysML v1 transformations, SysML v2-based transformations, and mappings from various modeling languages to AAS, no prior work has systematically addressed the transformation between SysML v2 and AAS. This gap is critical given the growing adoption of SysML v2 and the AAS standard for DTs. Existing work on SysML v1 has provided valuable methodologies for simulation integration and automation, but lacks compatibility with the enhanced expressiveness and interoperability of SysML v2. Similarly, prior research on integrating modeling languages with AAS has focused on conceptual mappings rather than automated model transformations. Our work addresses this gap by introducing the first automated model transformation between SysML v2 and AAS, enabling seamless integration of SysML v2 models into AAS-based digital twin ecosystems.

3. Research methodology

Our research methodology follows a structured approach to integrate SysML v2 and AAS through model-driven engineering techniques. The methodology comprises four main phases: (i) analysis of SysML v2 and AAS specifications, (ii) development of conceptual mappings between SysML v2 and AAS metamodel elements, (iii) implementation of model transformations to automate the translation process, and (iv) validation of the transformation through systematic testing. Through this approach, we establish a set of conceptual mappings (Tables 1–3), and model transformations executing a translation (Lúcio et al., 2016) between concepts defined by the SysML v2 and AAS metamodels. As a result, a SysML v2 model can be translated into an AAS model according to the elicited mappings, ensuring that both SysML v2 and AAS models can be utilized as DT models in model-driven digital twinning processes. Fig. 1 illustrates the proposed structured approach.

Specifications analysis. The initial phase involved a comprehensive analysis of both the SysML v2 and AAS specifications. For SysML v2, we examined the latest version (Object Management Group, 2024b) to extract key elements and their relationships for modeling system structure and behavior. For AAS, we analyzed the current metamodel published by IDTA-Workstream "Specification of AAS" (2025). Both SysML v2 and AAS specifications are descriptive artifacts, presenting the abstract syntax of their respective modeling concepts in a technology-independent manner. SysML v2, following standard OMG practice, adopts the Meta Object Facility (MOF) for describing its abstract syntax, while AAS employs a UML class diagram notation.

This analysis revealed significant differences in the expressiveness and scope of the two languages. Regarding language scope, SysML v2 is

“a general-purpose modeling language for modeling systems that is intended to facilitate a model-based systems engineering (MBSE) approach to engineer systems [...] representing the requirements, structure, and behavior of the system” (Object Management Group, 2024b), while AAS aims “to enable the meaningful exchange of information about assets and 14.0 components between partners in a value creation network” (IDTA-Workstream “Specification of AAS”, 2025).

SysML v2 is explicitly designed as a modeling language covering multiple facets of an engineered system, including its structure, behavior, requirements, analysis cases, and viewpoints (Bajaj et al., 2022), supported by precise semantics inherited from KerML (Object Management Group, 2024a; Almeida et al., 2025). In contrast, AAS targets a specific domain, namely the manufacturing domain, and focuses on exchanging structural asset information with limited and experimental support for behavioral aspects such as operations, events, and payloads, primarily when implementing Type 3 AAS models.

The specifications analysis phase concluded with the identification of a concrete technological space (TS) (Bézivin, 2006; Djurić et al., 2006) for SysML v2 and AAS specifications. We selected the Eclipse Modeling Framework (EMF), the de facto MOF implementation, as our technical foundation. MOF and Ecore establish a three-level metamodel architecture: M3, the meta-metamodel that defines the language for metamodels, i.e., MOF and Ecore; M2, metamodels (e.g., KerML and SysML v2) specifying modeling constructs; and M1, user models that instantiate M2 to represent domain-specific artifacts. The Eclipse platform offers a comprehensive tool ecosystem supporting model transformations (e.g., via QVT Object Management Group, 2016; Eclipse Foundation, 2025b) and model validation (e.g., via OCL) (Object Management Group, 2014). The SysML v2 metamodel (sysml.ecore) is provided by the Reference Implementation (Object Management Group, 2022). The AAS metamodel (aas.ecore) was adapted from our previous publication (Ferko et al., 2024), which extends the AAS metamodel from Dhouib et al. (2023). Fig. 2 illustrates the documents, artifacts, and their relationships arranged across metalayers.

Concept mappings. The mapping from SysML v2 to AAS follows an iterative process that systematically maps each SysML v2 modeling concept to one or more AAS concepts, where applicable. The outcomes are conceptual mappings for structural (see Table 1) and behavioral (see Table 2) aspects, as well as relationships (see Table 3). Following established practice in model-driven engineering (Berardinelli et al., 2016; Object Management Group, 2023), conceptual mappings provide technology-independent guidance for the model transformation development process, ensuring systematic translation between the two languages. Building upon our previous work (Ferko et al., 2024), we extended the mappings to include SysML v2 behavioral concepts for state machine modeling (state, transition, event, constraint, action).

The concept mapping process consisted of two phases. First, we identified straightforward direct mappings between SysML v2 concepts and

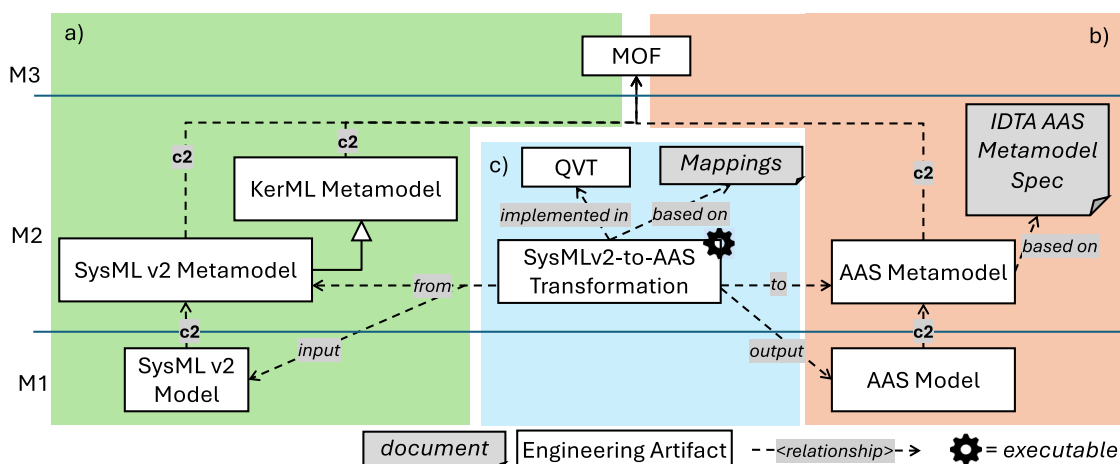


Fig. 2. Layered language integration architecture between SysML v2 and AAS via model transformations.

AAS. When direct mapping was not feasible due to differences in expressiveness, we developed a target pattern of AAS model elements that leverages AAS's native *Extension* and *Qualifier* concepts. [Section 5](#) provides detailed information on these mapping strategies.

Model transformation development. To automate the translation of SysML v2 models into AAS models, we implemented a model transformation in QVT (Object Management Group, 2016). The transformation was implemented following an iterative test-driven development approach using a test suite comprising 24 SysML v2 example models. The tests have been selected from a list of 100 SysML v2 examples provided by SysML v2 reference implementation (Object Management Group, 2022) for validation purposes. The development process focused on three key aspects:

- Developing queries to extract SysML v2 model elements for transformation.
- Implementing the mapping rules in QVTo, based on the conceptual mappings.
- Ensuring hierarchical structure preservation through recursion to maintain information and structural integrity.

When discrepancies were observed between the expected and generated AAS models, the transformation rules were iteratively refined until the intended mappings were accurately reflected. [Section 6](#) provides a comprehensive discussion of this development process.

Validation. Following a test-driven development approach, we conducted incremental and systematic validation of the AAS models generated by the model transformation against the AAS metamodel. Our test suite comprised SysML model examples from the SysML reference implementation (Object Management Group, 2022). We performed validation at two levels: (i) at a concept level, where each new SysML v2 metaclass (see Fig. 4) was individually tested to ensure proper transformation, and (ii) through end-to-end assessment using representative system models from the SysML v2 reference implementation (Object Management Group, 2022) to verify the transformation’s correctness and completeness. Section 6 presents a detailed analysis of our validation approach and results.

4. Uncovering KerML, SysML v2 and AAS metamodels

To investigate the conceptual alignment between SysML v2 and AAS specifications, we first introduce their metamodels as implemented in Eclipse EMF (see Fig. 2). We first inspect KerML, the language SysML v2 is derived from by extension, then we discuss SysML and AAS metamodels with excerpts from corresponding models.

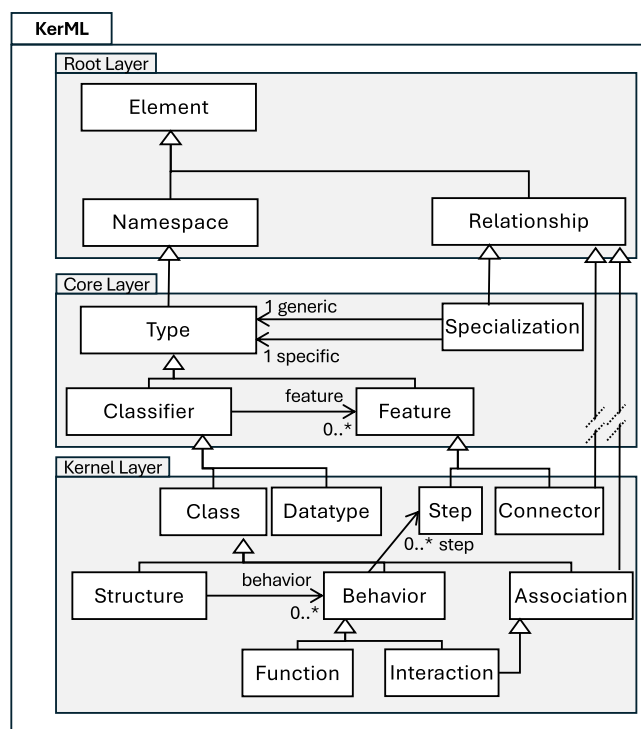


Fig. 3. Excerpt of KerML metamodel.

4.1. SysML v2, a KerML-based modeling language

KerML. KerML functions as a MOF-based general-purpose modeling language designed to serve as a *core*, domain-independent foundation for constructing semantically rich and interoperable modeling languages designed using the MOF metalanguage.

Fig. 3 shows an excerpt of the KerML metamodel. The *KerML Root Layer* establishes fundamental modeling primitives including *Element*, *Relationship*, and *Namespace*. The *KerML Core Layer* introduces the essential classification mechanisms based on the taxonomy of *Types*, *Classifiers*, and *Features*. A *Type* classifies things in a modeled system. Taxonomies of things are defined by *Specialization* relationship. A *Classifier* is a *Type* that serves as the primary definitional construct, representing categories of elements that share common *Features*. A *Feature* classifies how things in a modeled system are related. Finally, the *Kernel Layer* provides concrete concepts to model a composite system

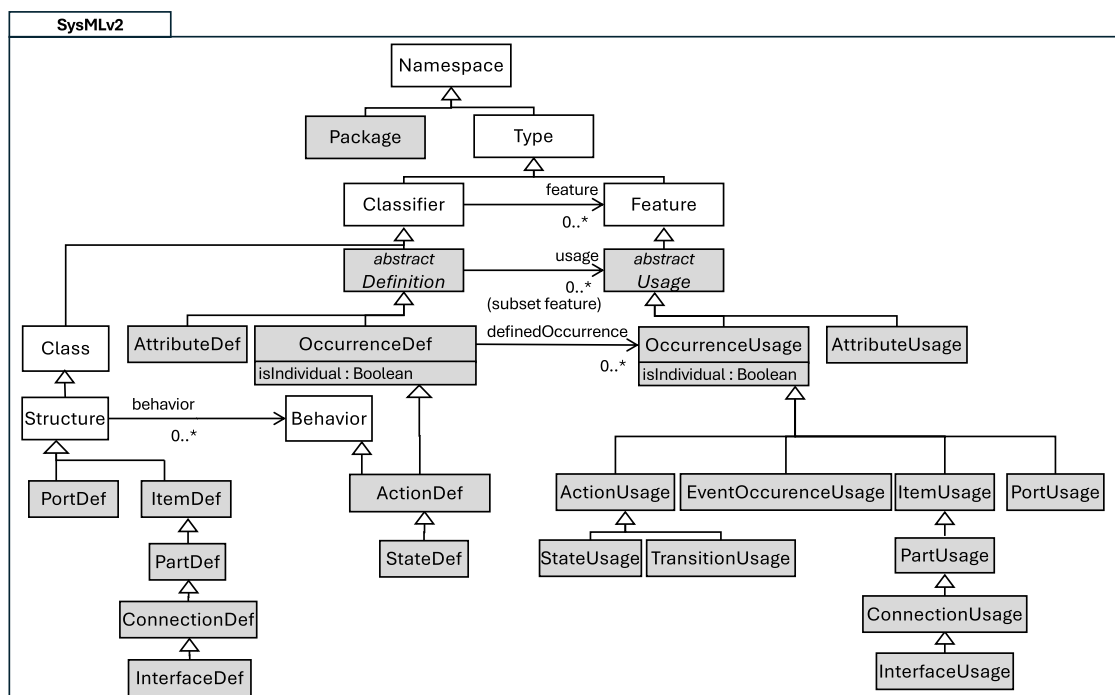


Fig. 4. Excerpt of SysML metamodel.

architecture (Class, Structure, Connector) and associated Behavior. Composite structures can be created by Classifiers bound through Connectors. A Behavior can be declared as a parameterized Function or associated with Steps featuring Classifiers. Finally, Interaction represents association among behaviors.

SysML v2. SysML v2 is the first language built on top of KerML by directly reusing or specializing its domain-neutral concepts. Fig. 4 shows an excerpt of the SysML v2 metamodel (grey) and how it extends KerML concepts (white).

SysML v2 organizes system representation across two distinct abstract concepts, i.e., Definition and Usage. Definition is an abstract metaclass that specializes the KerML Classifier. It establishes a classification *template* that captures the essential characteristics and capabilities of system element types and instances.

To support modularization and encapsulation, SysML v2 introduces Packages, which provide namespaces for organizing model elements. SysML v2 also emphasizes reusability by separating definitions and usages. A `Definition` classifies `Element`, defining its properties and structure, while a `Usage` represents a contextual instance of a `Definition`. Every `Usage` must reference at least one corresponding `Definition`, ensuring consistency and modularity across models. All the following SysML v2 concepts present a definition and usage. For brevity, any mention of a concept in the following text encompasses both its `Definition` and `Usage`.

SysML v2 treats every *thing* that exists in space and time as an Occurrence, and a system is a (possibly composite) occurrence that comprises other occurrences. As the root abstraction for such elements, an occurrence carries its own identity and unifies both structural (e.g., items, parts) and behavioral (e.g., interactions, actions) concepts under a single, precise taxonomy. An occurrence that represents an object with a unique identity is referred to as an individual occurrence (*isIndividual* set to true). *Item* are elements that flow through systems. A *Part* is a specialization of *Item* that specifies the structural components of a system, which can be composed and interconnected. A *Connection* specifies relationships between elements, such as parts or occurrences. A *Connector* is a connection between *Ports*. A *Port* is a well-structured interaction point between such elements. An *Interface* is a connection whose ends are restricted to be ports.

SysML v2 allows the modeling of behavior. In this paper, we consider Actions performed by a part, and States exhibited by a part under specific conditions. A State defines system behavior under specific conditions and may include entry, do, and exit actions. Actions associated with States regulate transitions between Behaviors. Transitions specify state changes, linking a source state to a target state⁴. EventOccurrenceUsage represents a specific occurrence of an event at a particular point in time or within a specific context. It enables event-driven modeling, supporting asynchronous messaging, triggers, and real-time interactions. Events are structured hierarchically and can influence state transitions.

SysML examples. To illustrate the structural modeling concepts in SysML v2, we consider the `VehicleDefinitions.sysml` example taken from [Object Management Group \(2022\)](#).

SysML primary notation is textual, keywords are highlighted (e.g., package). Note that there is no usage keyword in SysML v2's textual notation.

Listing 1 shows the structural definitions of a vehicle through part definitions (e.g., `part def Axle`) define reusable components of the vehicle. Ports (e.g., `port def DriveIF`) specify interaction points between components. For instance, the `Axle` is connected to the `Wheel` through two mounting points ports, both implementing the interface `Mounting`. This interface defines how torque is transferred as a flow from the axle (`AxleMountIF`) to the wheel (`WheelHubIF`) interfaces, enabling proper wheel rotation.

Listing 2 illustrates a state machine for vehicle operation, in SysML v2 taken from [Object Management Group \(2022\)](#). In this model, signals that trigger state transitions (e.g., `VehicleStartSignal`, `VehicleOnSignal`, `VehicleOffSignal`) are modeled as attributes. The state definition `VehicleStates` models the operational behavior of the vehicle, defining distinct states and transitions between them. Upon entry, the state machine starts in the `off` state. Transitions between the states are explicitly defined, where each transition is triggered by

⁴ TransitionUsage and EventOccurrenceUsage are usage-only elements in the SysML v2 metamodel, without corresponding definitions. This design choice reflects the nature of these elements as unique instances rather than reusable types.

```

1 package 'VehicleDefinitions' {
10 import ISQ.*;
15 part def Vehicle;
16 part def Transmission;
17 part def AxleAssembly;
18 part def Axle {
19     port leftMountingPoint:
        AxleMountIF;
20     port rightMountingPoint:
        AxleMountIF;
21 }
22 part def Wheel {
23     port hub: WheelHubIF;
24 }
25 part def Lugbolt {
26     attribute tighteningTorque
        :> ISQ::torque;
27 }

31 port def DriveIF {
32     in driveTorque:>ISQ::torque;
33 }
35 part def AxleMountIF {
36     out transferredTorque:>ISQ::
        torque;
37 }
39 port def WheelHubIF {
40     in appliedTorque:>ISQ::
        torque;
41 }
45 interface def Mounting {
47     end axleMount: AxleMountIF;
48     end hub: WheelHubIF;
49     flow axleMount.
        transferredTorque to hub.
        appliedTorque;}}

```

Listing 1. SysML v2 model excerpt with structural elements taken from [Systems Modeling Community \(2022\)](#)

```

1 package 'State Definition Example 12
    -1' {
2 attribute def VehicleStartSignal; 13 state starting;
3 attribute def VehicleOnSignal; 14 transition starting_to_on
4 attribute def VehicleOffSignal; 15     first starting
5 state def VehicleStates { 16     accept VehicleOnSignal
6     entry; then off; 17     then on;
7     state off; 18 state on;
8     transition off_to_starting 19     transition on_to_off
9         first off 20     first on
10     accept VehicleStartSignal 21     accept VehicleOffSignal
11     then starting; 22     then off;

```

Listing 2. SysML v2 model excerpt with behavioral elements taken from [Object Management Group \(2022\)](#)

one of the signal attributes. First transition, `off_to_starting` activates upon receiving `VehicleStartSignal`, `starting_to_on` follows `VehicleOnSignal`, and `on_to_off` returns the system to off when `VehicleOffSignal` is received.

4.2. AAS language specification

Fig. 5 shows an excerpt of the AAS metamodel⁵. The root concept is the `AssetAdministrationShell`, which encapsulates all information about an asset and is uniquely identified by its associated `AssetInformation`. An optional `derivedFrom` reference allows one to inherit or extend the contents of another. The `assetKind` attribute then records the ontological nature of the shell's subject—either a *type* (a class or template of assets) or an *instance* (a concrete physical or logical entity)—defaulting to `NotApplicable` when this distinction is not used. The `AssetAdministrationShell` metaclass may include multiple `Submodels`. Each `Submodel` organizes information relevant to specific domains or aspects of the asset, promoting modularization and separation of concerns. A `Submodel` can aggregate several `SubmodelElements` to represent asset details in a modular and structured way. Both `Submodel` and `SubmodelElement` specialize the abstract metaclass `HasKind` that features them with a `ModelingKind`. A template identifies a reusable blueprint to be replicated and customized when creating AAS models. An instance represents a completed, operational submodels or submodelements intended for direct utilization rather than further modification or replication.

The `SubmodelElement` metaclass is abstract and specialized by several key subclasses, each serving a specific purpose. Property encapsulates a single data attribute of an asset, including its value

and data type. Entity represents a physical or logical component of a system, further classified into *self-managed entity* and *co-managed entity* via the `EntityType`. A self-managed entity has its own `AssetAdministrationShell` while the latter is not independent and needs to be part of a self-managed entity. Operation defines actions associated with the asset, enabling operational behaviors and processes. A `BasicEventElement` enables the representation of event-driven behaviors and triggers. The `SubmodelElementCollection` allows logical grouping of submodel elements, aiding modularity and separation of concerns.

`RelationshipElement` models binary relationships between two referable elements. Relationships among AAS model elements are generic by design.

A `RelationshipElement` can represent is specifically designed to represent any and only binary relationships between a first and second `Referable` element. The `Referable` metaclass ensures that every element in the metamodel has a unique *idShort* to facilitate cross-referencing in AAS models. The *idShort* is referred to by the `Key` class, which enables the composition of complex relationships between referable elements, functioning as a navigation mechanism through the containment hierarchy.

The metamodel introduces two extension mechanisms. A `Qualifier` can be associated with any `Qualifiable` element, i.e., `Submodels` and `SubmodelElements`. The `Extension`, instead, represents a first-class modeling construct that can be independently associated with the `AssetAdministrationShell`, `Submodels`, and `SubmodelElements`. An `Extension` enables the incorporation of domain-specific or proprietary concepts that are not natively supported by the AAS metamodel. Both `Extension` and `Qualifier` consist of a type-value pair. The `Qualifier` attaches supplementary semantic information to its host element without altering the element's fundamental nature, while `Extension` is suitable for representing entirely new modeling concepts.

⁵ Colouring is for presentation only; colours carry no semantic meaning.

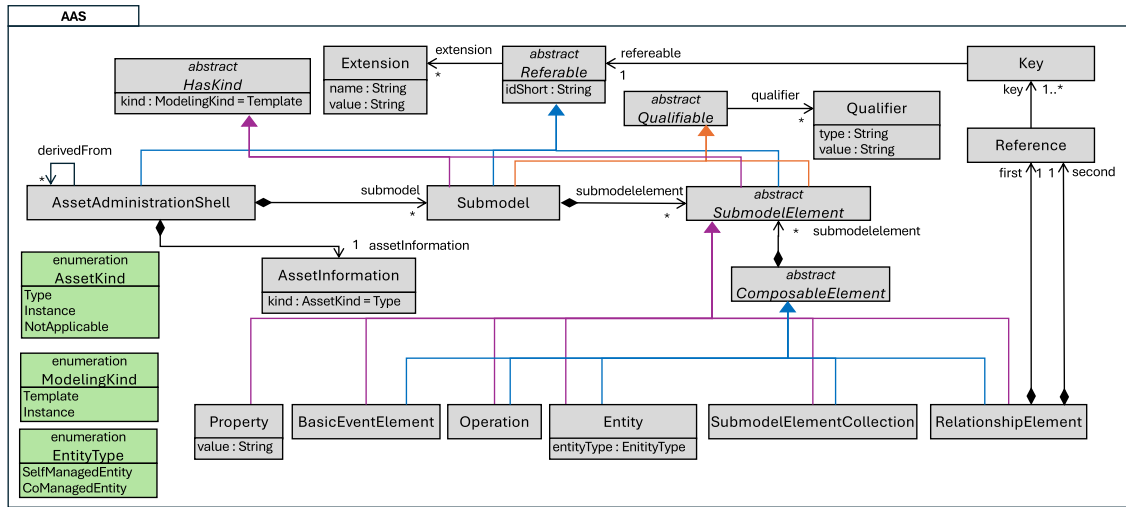


Figure 5. Excerpt of the AAS metamodel.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns:xsi="http://www.w3.org/2001/
  XMLSchema-instance" xmlns:AAS="www.jku.at/AAS">
3 <AAS:AasModelRoot name="ModelContainer">
4   <assetadministrationshells idShort="VehicleDefinitions">
5     <identification id="VehicleDefinition"/>
6     <assetInformation/>
7     <asset idShort="VehicleDefinition"/>
8     <submodel idShort="Hierarchical Structures">
9       <submodelelement xsi:type="AAS:Entity" idShort="Vehicle">
10        <qualifier type="PartDefinition"/>
11      </submodelelement>
12      <submodelelement xsi:type="AAS:Entity" idShort="Transmission">...
13    </submodelelement>
14    <submodelelement xsi:type="AAS:Entity" idShort="AxleAssembly">...
15  </submodelelement>
16  <submodelelement xsi:type="AAS:Entity" idShort="Axle">
17    <qualifier type="PartDefinition"/>
18    <submodelelement xsi:type="AAS:Entity" idShort="leftMountingPoint">
19      <qualifier type="PortUsage"/>
20      <submodelelement xsi:type="AAS:RelationshipElement" name="defined by">
21        <first name="leftMountingPoint">
22          <key referable="/0/@assetadministrationshells.0/@submodel.0/@submodelelement.3/
23            @submodelelement.0"/>
24        </first>
25        <second name="AxleMountIF">
26          <key referable="/0/@assetadministrationshells.0/@submodel.0/@submodelelement.7"/>
27        </second>
28      </submodelelement>
29    </submodelelement>
30  </submodelelement>
31  ...
160 </submodel>
161 </assetadministrationshells>
162 </AAS:AasModelRoot>
163 </xmi:XMI>

```

Listing 3. AAS model excerpt with structural elements

AAS examples. Listing 3 illustrates an AAS model representing a vehicle system in XMI, the EMF serialization format⁶. The model is hierarchically structured under *ModelContainer* (line 3), which serves as the root of the AAS model. Within this structure, the *VehicleDefinitions* shell contains a submodel called *Hierarchical Structures* (lines 4–8), which groups various vehicle components as entities. Core elements such as *Transmission*, *AxleAssembly*, and *Axle* are modeled as entities, while re-

lationships define dependencies between entities. For instance, the *leftMountingPoint* entity is associated with the *AxleMountIF* entity through a *RelationshipElement* (lines 21–29).

Listing 4 illustrates the representation of a state machine in AAS, modeling the operational behavior of a vehicle. The model is defined under the *State Definition Example-1* shell (line 3) and contains a submodel labeled *Operational Behavior* (line 4). Within this submodel, signals such as *VehicleStartSignal*, *VehicleOnSignal*, and *VehicleOffSignal* (lines 8–16) are represented as *Property* elements with *AttributeDefinition* qualifiers. State definition, *VehicleState*, is represented as an *Extension* (line 18). Individual states and transitions are encapsulated within *stateUsageCollection* (lines 21–33) and *transitionUsageCollection* (lines

⁶ The AAS examples include *Qualifier* and *Extension* elements that preserve SysML v2 annotations and semantics; the mapping approach is detailed in Section 5.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <AAS:AasModelRoot xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns:xsi="http://www.w3.org
  /2001/XMLSchema-instance" xmlns:AAS="www.jku.at/AAS" name="ModelContainer">
3   <assetadministrationshells idShort="State Definition Example-1">
4     <identification id="State Definition Example-1"/>
5     <assetInformation/>
6     <asset idShort="State Definition Example-1"/>
7     <submodel idShort="Operational Behavior">
8       <submodelelement xsi:type="AAS:Property" idShort="VehicleStartSignal">
9         <qualifier type="AttributeDefinition"/>
10      </submodelelement>
11      <submodelelement xsi:type="AAS:Property" idShort="VehicleOnSignal">
12        <qualifier type="AttributeDefinition"/>
13      </submodelelement>
14      <submodelelement xsi:type="AAS:Property" idShort="VehicleOffSignal">
15        <qualifier type="AttributeDefinition"/>
16      </submodelelement>
17      <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="state def ">
18        <extension name="VehicleState"/>
19        <qualifier type="StateDefinition"/>
20      </submodelelement>
21      <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="stateUsageCollection">
22        <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="s1">
23          <extension name="off"/>
24          <qualifier type="StateUsage"/>
25        </submodelelement>
26        ...
34      </submodelelement>
35      <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="transitionUsageCollection">
36        <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="t1">
37          <extension name="off_to_starting"/>
38          <qualifier type="TransitionUsage"/>
39          <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="actionsCollection">
40            <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="a1">
41              <qualifier type="ActionUsage"/>
42              <submodelelement xsi:type="AAS:Operation" idShort="first off"/>
43            </submodelelement>
44            <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="a2">
45              <qualifier type="ActionUsage"/>
46              <submodelelement xsi:type="AAS:Operation" idShort="accept ">
47                <submodelelement xsi:type="AAS:Property" idShort="VehicleOnSignal" />
48              </submodelelement>
49            </submodelelement>
50            <submodelelement xsi:type="AAS:SubmodelElementCollection" idShort="a3">
51              <qualifier type="Action Usage"/>
52              <submodelelement xsi:type="AAS:Operation"/>
53            </submodelelement>
54          </submodelelement>
55          ...
124      </submodel>
125    </assetadministrationshells>
126  </AAS:AasModelRoot>

```

Listing 4. AAS model excerpt with state machine

35–56), with each state and transition represented as an Extension. Each transition further contains actions organized under *actionsCollection* (lines 39–53), where actions are explicitly defined as Operations.

5. Mapping SysML v2 to AAS

The transformation from SysML v2 to AAS necessitates a systematic mapping between the metamodel elements of both languages. This section delineates the conceptual correspondences that underpin the model transformation, addressing the semantic and structural disparities between the two specifications.

During the mapping definition phase, the analysis revealed that numerous SysML v2 elements lack direct counterparts in the AAS metamodel. This discrepancy arises from the differing levels of expressiveness and domain focus of the two languages: SysML v2 serves as a general-purpose systems modeling language designed for comprehensive model-based systems engineering across multiple domains, while AAS specifically targets the manufacturing and Industry 4.0 domain with emphasis on asset information exchange.

To bridge this gap, the mapping leverages two native AAS mechanisms: the Extension concept and the Qualifiable concepts. The Extension mechanism permits the incorporation of domain-specific attributes without modifying the core AAS metamodel structure, proving particularly essential for representing SysML v2 behavioral elements that fall outside the AAS specification's scope (*IDTA-Workstream "Specification of AAS", 2025*). The Qualifiable concept enables the attachment of semantic metadata to AAS elements through Qualifiers, facilitating fine-grained semantic preservation.

Organization of mappings. Tables 1–3 present the comprehensive mapping between SysML v2 and AAS metamodel elements. The mapping rules are distributed across three tables, organized according to their modeling aspects, with each table employing a hierarchical presentation that reflects the inheritance relationships among SysML v2 metaclasses.

The categorization distinguishes three primary modeling aspects: structural modeling (Table 1) encompasses system organization and component definitions; behavioral modeling (Table 2) addresses dynamic system aspects, including actions, states, and events; relationships (Table 3) capture various forms of dependencies and connections.

Table 1
SysML v2 to AAS mapping – structural modeling.

SysML v2 Metaclass	AAS Metaclass(es)	ModelingKind	Qualifier Type
Package	AssetAdministrationShell, AssetInformation, Asset	-	package
Definition (abstract)			
-- OccurrenceDefinition	Entity	template	occurrence def
-- ItemDefinition	Entity	template	item def
-- PartDefinition	Entity	template	part def
-- PortDefinition	Entity	template	port def
--AttributeDefinition	Property	template	attribute def
Usage (abstract)			
-- OccurrenceUsage	Entity	template	(ref) occurrence
-- ItemUsage	Entity	template	(ref) item
-- PartUsage	Entity	template	part
-- PortUsage	Entity	template	(ref) port
--AttributeUsage	Property	template	attribute

Table 2
SysML v2 to AAS mapping – behavioral modeling.

SysML v2 Metaclass	AAS Metaclass(es)	ModelingKind	Qualifier Type
Definition (abstract)			
__ActionDefinition	Operation	template	action def
Usage (abstract)			
__ActionUsage	Operation	template	(ref) action
Definition (abstract)			
__StateDefinition	Extension	template	state def
Usage (abstract)			
__StateUsage	Extension	template	state
Usage (abstract)			
-- EventOccurrenceUsage	BasicEventElement	template	event occurrence
__TransitionUsage	Extension	template	transition

5.1. Classification of mapping patterns

The analysis of the mapping relationships reveals four distinct patterns that characterize the transformation between SysML v2 and AAS:

- Direct mappings occur when semantic equivalence exists between elements in both metamodels, exemplified by the correspondence between `AttributeDefinition` and `Property`.
- Entity-based mappings address the transformation of structural elements, where multiple SysML v2 concepts converge to the AAS `Entity` metaclass, necessitating type differentiation through `Qualifiers`. Our mapping deliberately does not set the `entityType` attribute, which distinguishes between self-managed entities (having their own shell) and co-managed entities (existing only as part of a self-managed entity). This mapping design decision aligns with the goal of generating AAS model templates that require engineering review: the determination of whether an entity should be self-managed or co-managed depends on the specific deployment context and cannot be automatically inferred from the SysML v2 model alone. To support this flexibility, the `entityType` attribute is defined as optional in our AAS metamodel implementation, allowing engineers to make this classification during the post-transformation refinement phase.
- Extension-based mappings accommodate behavioral elements that lack native representation in AAS, utilizing the `Extension` concept that creates a separate namespace to preserve SysML v2 semantics.
- Relationship mappings warrant particular attention due to their uniform transformation pattern. All SysML v2 relationships, regardless of their specific semantics, are mapped to a composite AAS structure consisting of a `RelationshipElement` accompanied by two

Reference instances and two Key instances. This pattern reflects the binary nature of relationships in SysML v2 and the requirement to maintain explicit references to both relationship ends within the AAS framework. The `RelationshipElement` serves as the primary container, while the `Reference-Key` pairs encode the ends of a binary relationship.

5.2. Bridging the gap: the extension and qualifier mapping patterns

The systematic application of the AAS `Extension` and `Qualifier` patterns constitutes a critical design decision in the mapping approach. As anticipated in [Section 4.2](#), the `Extension` and `Qualifier` concepts in AAS serve distinct purposes within the AAS metamodel, though both provide mechanisms for enhancing the expressiveness of AAS models beyond the core specification. We leveraged both concepts to cope with transformations between SysML v2 and AAS metamodels characterized by different expressiveness levels.

`Extension` allows adding a standalone model element within the AAS model. In the context of the SysML v2 to AAS transformation, `Extensions` are employed to represent behavioral elements such as states, transitions, and constraints that have no direct equivalent in the AAS metamodel.

`Qualifier`, in contrast, operates as a metadata annotation mechanism rather than an independent model element. In the transformation approach, `Qualifiers` serve the critical function of preserving the original SysML v2 `Type` (and specialization) information when mapped to generic AAS constructs.

The combined pattern emerges as a response to two distinct but related challenges in the transformation. First, the absence of behavioral modeling constructs in the AAS metamodel necessitates a mechanism for representing SysML v2 behavioral elements. Second, the many-to-one mapping relationships prevalent in the transformation require preservation of type information to maintain semantic integrity.

The design adheres to three principles. First, conceptual completeness ensures that all SysML v2 elements find representation in the AAS model, whether through direct mapping, qualification, or extension. Second, semantic preservation maintains the modeling intent and contextual information from the source SysML v2 model within the constraints of the target metamodel. Third, the approach ensures that sufficient structural information is retained to enable accurate reconstruction of the original SysML v2 model for reverse model transformations.

5.3. Assessment of the generated AAS model

The proposed mappings generate an AAS composed exclusively of template elements by setting the `ModelingKind` property of each element to `template`. This design choice is based on a specific rationale:

Table 3
SysML v2 to AAS mapping – relationships.

SysML v2 Metaclass	AAS Metaclass(es)	ModelingKind	Qualifier Type
Definition (abstract)			
<code> -- ConnectionDefinition</code>	RelationshipElement, Reference(2), Key(2)	template	connection def
<code> -- InterfaceDefinition</code>	RelationshipElement, Reference(2), Key(2)	template	interface def
Usage (abstract)			
<code> -- ConnectionUsage</code>	RelationshipElement, Reference(2), Key(2)	template	connection
<code> -- InterfaceUsage</code>	RelationshipElement, Reference(2), Key(2)	template	interface
Relationship (abstract)			
<code> -- Dependency</code>	RelationshipElement, Reference(2), Key(2)	template	dependency
<code> -- Specialization (:>)</code>	RelationshipElement, Reference(2), Key(2)	template	specialises
<code> -- Subsetting</code>	RelationshipElement, Reference(2), Key(2)	template	subsets
<code> -- Redefinition</code>	RelationshipElement, Reference(2), Key(2)	template	redefines
<code> -- Definition (:)</code>	RelationshipElement, Reference(2), Key(2)	template	defined by
<code> -- Connector</code>	RelationshipElement, Reference(2), Key(2)	template	connect

generating AAS templates to be assessed by AAS experts, who can determine whether to concretize the AAS model by switching the modeling kind to instance, provided sufficient technical information is available and compliance with AAS constraints is maintained. Notably, AAS enforces the constraint that template and instance elements cannot be mixed within the same submodel. This approach aligns with established practices in the AAS community. Many published Submodel Templates, such as IDTA's Digital Nameplate⁷, consist entirely of template elements.

The mapping generates a single top-level AssetAdministrationShell with AssetKind=Type from a SysML v2 Package, establishing a unified logical container for the generated AAS model. This design decision reflects the predominant modeling pattern in SysML v2 (and KerML), where a model represents a classification system. The choice of AssetKind=Type indicates that the AAS represents a class of assets rather than a particular physical instantiation. As a consequence, the proposed mapping generates an AAS type 1 that represents a design-time artifact. Therefore, a notable limitation of the current mapping concerns the runtime/instance level. SysML v2 supports the modeling of individual occurrences through OccurrenceDefinition and OccurrenceUsage elements with the *isIndividual* attribute set to true. These elements represent specific, identifiable instances in the physical world rather than component types. The present mapping does not consider AAS elements with AssetKind=Instance, thereby we are deliberately omitting the runtime/instance level.

6. Implementation and evaluation

We developed the model transformation using an iterative process inspired by the test-driven development in software engineering and aligned with the conceptual framework of model transformation testing and debugging proposed by Troya et al. (2023). This process incorporates multiple aspects of the conceptual framework by Troya et al. (2023), including *transformation definition* as a System Under Test (SUT), as well as testing approaches such as identifying a suitable set of *test models*, defining an *oracle function*, establishing *test adequacy criteria*, and applying structured *debugging techniques* to ensure correctness.

6.1. Transformation definition

We implemented the SysML v2 to AAS transformation using QVTo, a model-to-model transformation language designed for imperative rule-based transformations. The transformation process consists of two main steps: (i) extracting relevant elements from SysML v2 models using QVTo's *query* mechanism, and (ii) mapping these elements to corresponding AAS elements using QVTo's *mapping* rules.

Queries. Extracting elements from SysML v2 models is not straightforward due to the underlying KerML metamodel, which does not use hierarchical containment, but instead employs membership relationships to associate elements with namespaces and structures. This graph-based structure, rather than direct containment, requires explicit navigation of relationships such as *OwningMembership* and *FeatureMembership* to retrieve model elements. In Figs. 3 and 4, we have omitted the modeling concepts that SysML inherits from KerML to reduce complexity. Interested readers can explore KerML in more detail in Object Management Group (2024a). To define accurate queries, we examined the SysML v2 metamodel and analyzed models from the official SysML v2 repository to determine the correct traversal paths. For example, Listing 5 presents a QVTo query that retrieves all packages from the root element. This query navigates the *OwningMembership* relationship while filtering out annotations, ensuring that only valid packages are extracted.

Mappings. Once elements are extracted, the next step is defining QVTo mapping rules based on the conceptual correspondences explained in Section 5. For instance, Listing 6 illustrates the QVTo mapping rule for transforming SysML v2 elements (e.g., *Part*, *Port*) to AAS Entity elements.

This mapping function assigns a unique identifier to the resulting AAS entity using the *idShort* attribute (line 2), ensuring that each entity can be consistently referenced within the AAS model. A *Qualifier* is created (lines 3–6) to store the type extracted from the *owningMembership* relationship.

Structural integrity. To determine whether a structure-preserving morphism exists between the two metamodels, we examine how the containment hierarchies in SysML v2 can be represented in AAS. In SysML v2, structural elements such as *ItemDefinition*, *PartDefinition*, and *PortDefinition*, along with their corresponding usages (*ItemUsage*, *PartUsage*, and *PortUsage*), form nested containment relationships as shown in Fig. 4.

Our transformation maps all these elements to AAS Entity elements, which inherit from *SubmodelElement* in the AAS metamodel (Fig. 5). Since *SubmodelElement* instances can contain other *SubmodelElement* instances through the *submodelElement* association, the AAS metamodel supports arbitrary nesting of entities. This structural correspondence allows us to preserve the containment hierarchy of SysML v2 elements: a *PartDefinition* containing *PortDefinition* elements in SysML v2 is transformed into an Entity (qualified as type=part) containing nested Entity elements (qualified as type=port) in AAS. Specifically, owned features are transformed into nested AAS entities using the *feature2Entity()* function (lines 7–8), which also recursively maps nested features. Meanwhile, owned attributes are converted into property collections via *toSubmodelElementCollection4Property()*. Other mapping functions follow a similar pattern.

⁷ <https://industrialdigitaltwin.org/en/content-hub/submodels>


```

1 //Query for Namespace
2 //return packages to be mapped with Asset administration shells
3 query SysMLv2MM ::Element::ownedPackages(): Sequence(SysMLv2MM ::Package)
4 {
5   return self.ownedRelationship[SysMLv2MM ::OwningMembership]
6   [ownedRelatedElement->forAll(not oclIsKindOf(AnnotatingElement))].
   ownedRelatedElement[Package];
7 }

```

Listing 5. QVTo query to retrieve all packages

```

1 mapping SysMLv2MM::Element::toEntity(): AASMM::Entity{
2   result.idShort := self.id;
3   var qualifier := object AASMM::Qualifier {
4     type := self.owningMembership.memberElement.type;
5   };
6   result.qualifier := qualifier;
7   result.submodelelement += self.ownedFeatures() -> map feature2Entity();
8   result.submodelelement += self.ownedAttributes -> map
   toSubmodelElementCollection4Property();
9   result.submodelelement += self.ownedComments()[Documentation] -> map
   toSubmodelElementCollection4ExtensionorDocument();}

```

Listing 6. QVTo mapping for AAS Entity

Our metamodel analysis confirms that for the structural elements within our transformation scope, the containment paths from root to leaf elements can be faithfully represented in AAS, ensuring that the hierarchical organization and structural semantics of the original SysML v2 model are preserved throughout the transformation process.

6.2. Transformation testing

Test model suite. To ensure the correctness and completeness of the transformation, we evaluated the model transformation using a systematic test suite. The test suite consists of two levels of evaluation: (i) transformation testing for each SysML v2 metaclass that is not abstract in the metamodel, and (ii) a more complex end-to-end evaluation with a representative system model. For the first level, we selected example models that instantiate concrete metaclasses from the SysML v2 metamodel. In total, we considered 24 examples sourced from the official SysML v2 repository (Object Management Group, 2022) out of a total of 100 examples, organized by metaclass⁸. Running the transformation on these individual examples allowed us to verify that all relevant SysML v2 elements were successfully mapped to AAS elements. The second level of testing involved applying the transformation to a more comprehensive system model that integrates multiple structural and behavioral elements. This model, an excerpt of which is shown in Listing 1, includes various part definitions, ports, and interfaces. The corresponding AAS model generated from this transformation is depicted in Listing 3, demonstrating the hierarchical structuring and the correct mapping of elements within the AAS framework. For example, a SysML v2 Part Definition (e.g., Axle, Wheel, Lugbolt in Listing 1) is transformed into an AAS Entity (Fig. 3) with qualifiers that denote the type of the element in SysML v2.

Oracle function. For the first level of evaluation of the test suite, since no predefined expected models were available to serve as an oracle, we defined our oracle criteria based on information preservation and structural integrity preservation. For the second level of evaluation, instead, we compared the transformation results against the expected output, hence, manually created models. When creating the expected output, we still ensured that the structural integrity, hierarchical organization, and semantic correctness of the transformed model align with the conceptual mappings.

Test adequacy criteria. We used two main test adequacy criteria, being *metamodel coverage* and *rule coverage analysis*. Metamodel coverage

ensures that all relevant SysML v2 elements are represented in the test cases, verifying that each transformation rule is executed with at least one input model covering its corresponding SysML v2 metaclass. Rule coverage analysis systematically checks that every transformation rule is triggered at least once during testing, ensuring that all mapping functions are exercised under different scenarios to detect potential gaps in the transformation logic. By combining fine-grained unit tests for individual SysML v2 classes with system-level validation on a complex model, we achieve comprehensive assurance that the transformation operates correctly across various modeling scenarios.

6.3. Debugging

During development and validation, we applied both *static* and *dynamic* debugging techniques to detect and resolve transformation errors.

Static debugging. We verified compliance with the SysML v2 and AAS metamodel constraints and ensured rule consistency. Constraint validation checked whether the transformation adhered to structural requirements, while rule consistency analysis identified incorrect mappings and structural misalignments.

Dynamic debugging. We monitored transformation execution and analyzed trace models to track how SysML v2 elements mapped to AAS entities. *Trace model debugging* allowed us to follow element mappings step by step, while *differential debugging* helped us compare transformation outputs before and after fixes. We identified and addressed several common transformation issues. *Relationship misalignment*, caused by incorrect membership resolution, was resolved by refining query filters. *Hierarchy loss*, where nested elements were not correctly transferred, was mitigated by implementing recursive mappings.

7. Discussion

This section provides a critical discussion of the proposed mapping between SysML v2 to AAS, its implementation, limitations, and future work.

Transformation scope. The proposed mappings focus solely on architectural concerns, i.e., structural and behavioral aspects of SysML v2, excluding requirements, viewpoints, and analysis cases (Object Management Group, 2024b; Bajaj et al., 2022), which are not covered by AAS and, thus, would demand extensive use of Extension and Qualifier mapping patterns, which is left as future work. From an AAS standpoint, the proposed mappings aim at generating AAS Type 1 templates from SysML v2 including structural and behavioral information. If the transformation is applied to valid SysML v2 models containing elements

⁸ <https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation/tree/master/sysml/src/training>

outside the considered metamodel fragment, it will still attempt to generate the AAS representation, but these unsupported elements will simply be ignored. This ensures that the transformation remains functional and adaptable, even when processing models that extend beyond the targeted subset of SysML v2.

Structural preservation mechanisms. Structural preservation refers to retaining the containment hierarchy of SysML v2 structural elements mapped to entities within the generated AAS model (see [Table 1](#)).

To determine whether a structure-preserving morphism exists between the two metamodels, we compare the containment paths from the root metaclass to the leaf metaclasses in both languages. By analyzing these containment relationships, we verify that the structural mappings between SysML v2 and AAS preserve the nested organization of SysML v2 definitions and usages through nested entities. Our metamodel analysis and the defined mappings show that the containment paths for the overlapping structural elements in SysML v2 and AAS are isomorphic, ensuring that structural integrity is preserved during transformation. This approach ensures that the hierarchical organization of the original SysML v2 model is retained within the AAS representation, thereby preserving essential structural semantics and supporting traceability throughout the transformation process.

Semantic alignment and theoretical foundations. A central aspect of our transformation approach is the conceptual mapping between SysML v2 and AAS metamodel elements and their operational implementation in an executable QVT artifact. While our model transformation approach guarantees syntactic integrity by design—ensuring that all generated AAS models conform to the AAS metamodel, we acknowledge that syntactic validity alone does not imply semantic equivalence between the source and target models. However, in model transformation scenarios involving heterogeneous languages like SysML v2 and AAS, full semantic equivalence is rarely achievable, nor is it typically the goal. Instead, the transformation serves as a semantic bridge between languages with different purposes and levels of expressiveness. In this work, the goal is to empower the digital twin modeling by combining SysML v2 and AAS.

We acknowledge the absence of a formal verification of the proposed mappings (e.g., [Ulitin et al., 2019](#)), which goes beyond the scope of this work. Our main goal is to establish an interpretable and implementation-agnostic alignment between their core modelling constructs. This aligns with established practices in MDE, where transformation logic is typically guided by conceptual mappings derived from domain knowledge and metamodel analysis ([Object Management Group, 2023](#); [Berardinelli et al., 2016](#)). As such, the conceptual mappings play a dual role: they inform the QVT implementation of the transformation and provide a reusable reference for others aiming to bridge SysML v2 and AAS. By grounding these mappings in metamodel relationships rather than in language-specific constraints, we ensure flexibility, clarity, and adaptability of the transformation logic without requiring a formal semantic translation framework.

Bidirectional transformation design. Our conceptual mappings are designed to enable bidirectional model transformations, facilitating the conversion of AAS models back into SysML v2 models following potential modifications. Such bidirectional transformations are particularly beneficial in round-trip engineering scenarios, where updates made in AAS representations can be systematically reflected in the original SysML v2 models, thereby ensuring consistency across engineering phases. Preserving fidelity in both transformation directions significantly enhances the practical utility of our approach, supporting seamless interoperability between SysML v2 and AAS. In this work, however, we have implemented only the forward transformation from SysML v2 to AAS, leaving the reverse transformation as future work.

Limitations and integration strategies. While our transformation ensures a conceptual alignment between SysML v2 and AAS, it is important to acknowledge limitations arising from the expressiveness gap between the two standards. AAS, by design, does not support behavioral modelling constructs. As a result, scenarios requiring rich behavioral speci-

fications cannot be fully addressed using AAS alone. To overcome this limitation, our approach extends the AAS metamodel through custom Qualifiers to encode behavioral distinctions. However, such reliance on extension mechanisms can lead to reduced semantic transparency and diminished interoperability, particularly when used outside controlled toolchains.

This limitation is especially relevant in DT use cases involving functional or non-functional analysis, such as simulation, performance estimation, or test case generation, which typically require behaviorally complete models. In these contexts, AAS cannot serve as the sole DT model. This raises a broader question around integration strategy: what is the appropriate trade-off when linking AAS with other modelling languages? Should the entire source model be translated into AAS, only selected parts, or should the AAS simply reference external models expressed in a more expressive Domain-Specific Language (DSL)? There is no one-size-fits-all solution. The right choice depends on the intended use of the AAS and the type of engineering information that must be extracted from it.

Extended validation of AAS models. The validation of the generated AAS model in this study focuses on its structural conformance with the Ecore-based implementation of the IDTA AAS metamodel specification. Although the metamodel specification by [IDTA-Workstream "Specification of AAS" \(2025\)](#) includes additional semantic constraints expressed textually, these constraints are not enforced in our current approach. In future work, we expect to address this gap by formalizing those requirements as Object Constraint Language (OCL) invariants.

Serialization format and deployability. The AAS specification ([IDTA-Workstream "Specification of AAS", 2025](#)) defines multiple standardized serialization syntaxes, notably a JSON encoding, an XML encoding, and a JSON-LD variant to support linked-data scenarios. In our implementation, we leverage the Eclipse Modeling Framework's native XMI serialization to persist the generated AAS models. While XMI delivers seamless integration with EMF-enabled tools, it is not recognized as a deployment format by runtime environments such as the BaSyx AAS server ([Eclipse Foundation, 2025a](#)). Consequently, a dedicated serialization step is required to convert the EMF-based XMI files into one of the official AAS formats (e.g., AAS-JSON or AAS-XML) before registering the shell with an AAS server. Integrating this intermediate transformation into the toolchain is part of our future work.

Linguistic and ontological instantiation in SysML v2 and AAS. Beyond the immediate extensions to our transformation approach, an interesting area for future investigation is the systematic comparison of instantiation mechanisms between SysML v2 and AAS. While our current work addresses the practical mapping between these languages for design models, we have observed that they employ fundamentally different approaches to linguistic and ontological instantiation, which would be worth investigating for modeling the runtime of systems. SysMLv2 adopts a unified Definition/Usage pattern that combines both dimensions, whereas AAS provides an explicit two-dimensional matrix through its `AssetKind` and `ModelingKind` attributes. This difference has profound implications for model semantics and transformation completeness. Future work will investigate how these distinct instantiation dimensions affect the preservation of modeling intent during transformation, potentially leading to more sophisticated mapping strategies that can fully leverage the expressiveness of both languages.

Performance considerations. Although the focus of this work was not on validating the performance of the transformation in terms of execution time or scalability—but rather on introducing the first automated model transformation between SysML v2 and AAS—we recognize that performance considerations are crucial for practical adoption, especially given the high industrial relevance of such a transformation. To provide initial evidence on the performance of the transformation and its potential scalability, we opportunistically recorded the execution times for each transformation run across the 24 models in our test suite.

The transformation was executed on a laptop equipped with an Apple M1 Pro-chip and 16GB of memory. Overall, we observed execution

times ranging from milliseconds to a few seconds, depending on the complexity of the models involved. We omit the exact measurements, as we noted that different executions of the transformation occasionally resulted in slightly varying execution times, likely due to background tasks running on the laptop. Despite these variations, the recorded times provide an initial indication of the transformation's computational feasibility and suggest that the approach can handle models of varying complexity within a reasonable execution time. Considering that this transformation is non-time-critical and given that the current state of practice relies on manual translation of these models, we argue that the execution time of our automated transformation is negligible in comparison to the existing benchmark. The automation introduced by our approach significantly reduces the time and effort required for model translation, making execution performance a secondary concern relative to the efficiency gains and accuracy improvements achieved through automation. Thus, we leave further performance optimizations of the transformation as subject for future work.

Threats to validity. While our evaluation demonstrates the correctness of the transformation with respect to information and structural preservation, we acknowledge a key threat to validity may lie in the lack of publicly available ground truth models. This is largely because such models are often proprietary in industrial settings and thus not disclosed, while academic examples typically remain illustrative and limited in scope.

We also recognize that using manually created reference models to validate our transformation may raise concerns about potential bias, as these models were crafted following the same mapping logic. However, we emphasize that the intent of model transformation research is not to invent arbitrary mappings, but to formalize known correspondences derived from domain expertise and standard specifications. The primary goal is to automate what would otherwise be a time-consuming and error-prone manual process.

In this work, we followed best practices from the model transformation testing literature, particularly the guidelines proposed by Troya et al. (2023), which recommend defining oracle functions based on semantic properties (such as preservation of information and structure), and evaluating transformation adequacy through metamodel and rule coverage. Indeed, our adequacy criteria were based on the coverage of the SysML v2 metamodel as well as on the transformation rules coverage. Specifically, we aimed for our transformation to encompass at least all the elements illustrated in Fig. 4. In addition, we ensured that each transformation rule was executed at least once across the test suite models. Ensuring comprehensive coverage of these elements was essential for validating the correctness of our transformation approach.

8. Conclusion and future work

In this paper, we proposed a model-driven approach for generating AAS models from SysML v2, enabling a more efficient engineering of DTs through model reuse. We established a conceptual mapping between core structural and behavioral modeling elements of both languages. To implement this mapping, we adopted an iterative test-driven development approach and developed a model transformation using the QVT transformation language. This implementation incorporates structured queries for SysML v2 element extraction and transformation rules into AAS elements that preserve both hierarchical structure and static semantic integrity.

As part of the iterative, test-driven development, we validated the transformation on a diverse set of 24 models sourced from the official repository of SysML v2. The results demonstrated that the transformation accurately generates AAS models, ensuring correctness and consistency throughout the transformation process. While our approach successfully bridges the syntactic gap between SysML v2 and AAS, we acknowledge that the expressiveness differences between the two standards, particularly regarding behavioral modeling, present inherent lim-

itations that must be considered when applying the transformation in practice.

As part of future work, we plan to extend the mappings to increase the coverage of SysML v2 metamodel. In particular, we aim at addressing the ontological instantiation aspect by including the mapping of SysML v2 individual occurrences and instance assets (i.e., `AssetKind = instance`) in AAS. Another promising direction is enabling bidirectional transformations, allowing modifications in AAS models to be propagated back to SysML v2 models. Additionally, we plan to: (i) formalize additional AAS validation constraints using OCL to ensure semantic consistency beyond structural conformance, (ii) investigate hybrid approaches that integrate AAS with domain-specific or general-purpose languages to overcome behavioral modeling limitations, and (iii) conduct scalability and performance evaluations to assess the transformation's applicability to large-scale system models. These extensions will further enhance the practical utility of our approach in industrial digital twin engineering scenarios.

CRedit authorship contribution statement

Enxhi Ferko: Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization; **Luca Berardinelli:** Writing – original draft, Supervision, Methodology, Conceptualization; **Alessio Bucaioni:** Writing – original draft, Supervision, Methodology, Investigation, Conceptualization; **Moris Behnam:** Supervision, Methodology, Conceptualization; **Manuel Wimmer:** Writing – original draft, Supervision, Methodology.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work has received funding from the Key Digital Technologies Joint Undertaking (KDT-JU) through the project MATISSE, grant agreement No. 101140216.

References

- Almeida, J. P.A., Ferreira Pires, L., Guizzardi, G., Wagner, G., 2025. An analysis of the semantic foundation of kerML and sysML v2. In: Maass, W., Han, H., Yasar, H., Multari, N. (Eds.), *Conceptual Modeling*. Springer. 15238, pp. 133–151. https://doi.org/10.1007/978-3-031-75872-0_8
- Bajaj, M., Friedenthal, S., Seidewitz, E., 2022. Systems modeling language (sysML v2) support for digital engineering. *Insight* 25 (1), 19–24. <https://doi.org/10.1002/inst.12367>
- Berardinelli, L., Biffi, S., Lüder, A., Mätzler, E., Mayerhofer, T., Wimmer, M., Wolny, S., 2016. Cross-disciplinary engineering with automationML and SysML. *At-Automatisierungstechnik* 64 (4), 253–269.
- Brambilla, M., Cabot, J., Wimmer, M., 2017. *Model-Driven Software Engineering in Practice*. Synthesis Lectures on Software Engineering, Springer. <https://doi.org/10.1007/978-3-031-02549-5>
- Burgueño, L., Ciccozzi, F., Famelis, M., Kappel, G., Lambers, L., Mosser, S., Paige, R.F., Pierantonio, A., Rensink, A., Salay, R., Taentzer, G., Vallecillo, A., Wimmer, M., 2019. Contents for a model-based software engineering body of knowledge. *Softw. Syst. Model.* 18 (6), 3193–3205. <https://doi.org/10.1007/s10270-019-00746-9>
- Bézivin, J., 2005. On the unification power of models. *Softw. Syst. Model.* 4 (2), 171–188. <https://doi.org/10.1007/s10270-005-0079-0>
- Bézivin, J., 2006. Model driven engineering: an emerging technical space. In: *Generative and Transformational Techniques in Software Engineering*. Springer. 4143, pp. 36–64. https://doi.org/10.1007/11877028_2
- Cavaliere, S., Salafia, M.G., 2020. Insights into mapping solutions based on OPC UA information model applied to the industry 4.0 asset administration shell. *Computers* 9 (2), 28. <https://doi.org/10.3390/computers9020028>

- Combemale, B., Kienzle, J., Mussbacher, G., Ali, H., Amyot, D., Bagherzadeh, M., Batot, E., Bencomo, N., Benni, B., Bruel, J.-M., Cabot, J., Cheng, B. H.C., Collet, P., Engels, G., Heinrich, R., Jezequel, J.-M., Koziol, A., Mosser, S., Reussner, R., Sahraoui, H., Saini, R., Sallou, J., Stinckwich, S., Syriani, E., Wimmer, M., 2021. A hitchhiker's guide to model-driven engineering for data-centric systems. *IEEE Softw.* 38 (4), 71–84. <https://doi.org/10.1109/MS.2020.2995125>
- Czarnecki, K., Helsen, S., 2006. Feature-based survey of model transformation approaches. *IBM Syst. J.* 45 (3), 621–645. <https://doi.org/10.1147/sj.453.0621>
- Dalibor, M., Jansen, N., Rumpe, B., Schmalzing, D., Wachtmeister, L., Wimmer, M., Wortmann, A., 2022. A cross-domain systematic mapping study on software engineering for digital twins. *J. Syst. Softw.* 193, 111361. <https://doi.org/10.1016/j.jss.2022.111361>
- Dhouib, S., Huang, Y., Smaoui, A., Bhanja, T., Gezer, V., 2023. Papyrus4manufacturing: a model-based systems engineering approach to AAS digital twins. In: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, Sinaia, Romania, pp. 1–8. <https://doi.org/10.1109/ETFA54631.2023.10275523>
- DIN SPEC 91345:2016-04, 2016. Reference Architecture Model Industrie 4.0 (RAMI4.0). Last accessed on 2024-03-21. <https://www.beuth.de/en/technical-rule/din-spec-91345/250940128>
- Djurić, D., Gašević, D., Devedžić, V., 2006. The tao of modeling spaces. *J. Object Technol.* 5 (8), 125–147.
- Eclipse Foundation, 2025a. Eclipse BaSyx - Industry 4.0 Operating System. Accessed: 2025-02-24. <https://eclipse.dev/basyx/>
- Eclipse Foundation, 2025b. Eclipse QVT Operational. Accessed: 2025-03-02. <https://projects.eclipse.org/projects/modeling.mmt.qvt-oml>
- Eramo, R., Bordeleau, F., Combemale, B., Brand, M. V.D., Wimmer, M., Wortmann, A., 2022. Conceptualizing digital twins. *IEEE Softw.* 39 (2), 39–46. <https://doi.org/10.1109/MS.2021.3130755>
- Ferko, E., Berardinelli, L., Bucaioni, A., Behnam, M., Wimmer, M., 2024. Towards interoperable digital twins: integrating sysml into AAS with higher-order transformations. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C). IEEE, Hyderabad, India, pp. 342–349. <https://doi.org/10.1109/ICSA-C63560.2024.00063>
- Ferko, E., Bucaioni, A., Behnam, M., 2022. Architecting digital twins. *IEEE Access* 10, 50335–50350. <https://doi.org/10.1109/ACCESS.2022.3172964>
- Ferko, E., Bucaioni, A., Pelliccione, P., Behnam, M., 2023. Analysing interoperability in digital twin software architectures for manufacturing. In: Tekinerdogan, B., Trubiani, C., Tibermacine, C., Scandurra, P., Cuesta, C.E. (Eds.), *Software Architecture*. Springer, Cham, 14212, pp. 170–188. https://doi.org/10.1007/978-3-031-42592-9_12
- Ferko, E., et al., 2025. SysMLv2-AAS Integration. Accessed: 2025-06-29. <https://github.com/jku-win-se/SysMLv2-AAS-integration>
- Frysak, J., Kaar, C., Stary, C., 2018. Benefits and pitfalls applying RAMI4.0. In: 2018 IEEE Industrial Cyber-Physical Systems (ICPS). IEEE, St. Petersburg, pp. 32–37. <https://doi.org/10.1109/ICPHY.2018.8387633>
- Fuchs, J., Schmidt, J., Franke, J., Rehman, K., Sauer, M., Karnouskos, S., 2019. I4.0-compliant integration of assets utilizing the asset administration shell. In: 2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, Zaragoza, Spain, pp. 1243–1247. <https://doi.org/10.1109/ETFA.2019.8869255>
- Grieves, M., 2014. Digital Twin: Manufacturing Excellence Through Virtual Factory Replication. Technical Report 1. White paper.
- Guzina, L., Ferko, E., Bucaioni, A., 2022. Investigating digital twin: a systematic mapping study. In: Ng, A. H.C., Syberfeldt, A., Höglberg, D., Holm, M. (Eds.), *Advances in Transdisciplinary Engineering*. IOS Press, pp. 449–460. <https://doi.org/10.3233/ATDE220164>
- Heermann, H., Herzog, M., Koch, J., Grimm, C., 2024. Generating digital twins from SysMLv2 models. In: 2024 IEEE 22nd International Conference on Industrial Informatics (INDIN). IEEE, pp. 1–6.
- IDTA-Workstream "Specification of AAS", 2025. Specification of the Asset Administration Shell Part 1: Metamodel. <https://doi.org/10.62628/idta.01001-3-1>
- Industrial Digital Twin Association, 2025. IDTA – Der Standard für den Digitalen Zwilling. Accessed: 2025-02-24. <https://industrialdigitaltwin.org/>
- International Electrotechnical Commission, 2020. Industrial-process measurement, control and automation – life-cycle-management for systems and components. International Standard IEC 62890:2020, TC 65 Industrial-process measurement, control and automation. <https://webstore.iec.ch/en/publication/30583>
- International Organization for Standardization, 2021. Automation systems and integration — digital twin framework for manufacturing — part 1: Overview and general principles. ISO 23247-1:2021. <https://www.iso.org/standard/75066.html>
- Jacoby, M., Volz, F., Weisenbacher, C., Müller, J., 2022. FA3St service – an open source implementation of the reactive asset administration shell. In: 2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, Stuttgart, Germany, pp. 1–8. <https://doi.org/10.1109/ETFA52439.2022.9921584>
- Jansen, N., Pfeiffe, J., Rumpe, B., Schmalzing, D., Wortmann, A., 2022. The language of SysML v2 under the magnifying glass. *J. Object Technol.* 21 (3), 3:1. <https://doi.org/10.5381/jot.2022.21.3.a11>
- Jones, D., Snider, C., Nassehi, A., Yon, J., Hicks, B., 2020. Characterising the digital twin: a systematic literature review. *CIRP J. Manuf. Sci. Technol.* 29, 36–52. <https://doi.org/10.1016/j.cirpj.2020.02.002>
- Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W., 2018. Digital twin in manufacturing: a categorical literature review and classification. *IFAC-PapersOnLine* 51 (11), 1016–1022. <https://doi.org/10.1016/j.ifacol.2018.08.474>
- Luder, A., Behnert, A.-K., Rinker, F., Biffel, S., 2020. Generating industry 4.0 asset administration shells with data from engineering data logistics. In: 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, Vienna, Austria, pp. 867–874. <https://doi.org/10.1109/ETFA46521.2020.9212149>
- Lúcio, L., Amrani, M., Dingel, J., Lambers, L., Salay, R., Selim, G. M.K., Syriani, E., Wimmer, M., 2016. Model transformation intents and their properties. *Softw. Syst. Model.* 15 (3), 647–684. <https://doi.org/10.1007/s10270-014-0429-x>
- Miny, T., Heppner, S., Guo, W., Möller, L., Kleinert, T., 2023a. Standardized integration of source systems into asset administration shell realizations. In: 2023 IEEE 21st International Conference on Industrial Informatics (INDIN). IEEE, Lemgo, Germany, pp. 1–6. <https://doi.org/10.1109/INDIN51400.2023.10218162>
- Miny, T., Thies, M., Lukic, L., Käbisch, S., Oladipupo, K., Diedrich, C., Kleinert, T., 2023b. Overview and comparison of asset information model standards. *IEEE Access* 11, 99189–99221. <https://doi.org/10.1109/ACCESS.2023.3312286>
- Molina, M., Treichel, T., 2023. Achieving interoperability with MBSE and asset administration shells: integration of MATLAB/simulink and basyx. In: 2023 IEEE/ACM 11th International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems (SESOS). IEEE, Melbourne, Australia, pp. 1–4. <https://doi.org/10.1109/SESOS59159.2023.00005>
- Object Management Group, 2014. Object Constraint Language (OCL) Specification Version 2.4. Accessed: 2025-03-02. <https://www.omg.org/spec/OCL/2.4>
- Object Management Group, 2016. Qvt (query/view/transformation) specification version 1.3. Accessed: 2025-03-02. <https://www.omg.org/spec/QVT/1.3>
- Object Management Group, 2022. SysML-v2-Pilot-Implementation. <https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation>
- Object Management Group, 2023. SysML v1 to v2 Transformation. <https://www.omg.org/spec/SysML/2.0/Beta1/Transformation/PDF>
- Object Management Group, 2024a. Kernel Modeling Language (KerML) Specification (Version 1.0 beta 2). Available at <https://www.omg.org/spec/KerML/>
- xcollab[Object Management Group], 2024b. OMG Systems Modeling Language (SysML) Version 2.0 beta 2. Available at <https://www.omg.org/spec/SysML/>
- Pakala, H.K., Oladipupo, K.O., Käbisch, S., Diedrich, C., 2021. Integration of asset administration shell and web of things. *Kommunikation in der Automation (KommA)* <https://doi.org/10.25673/39570>
- Pereira Pessoa, M.V., Lopes Schuch, A.P., Bezerra, J.M., 2023. Using model-based systems engineering to design system-based digital twins. *INCOSE Int. Symp.* 33 (1), 1435–1449. <https://doi.org/10.1002/iis2.13091>
- Platenius-Mohr, M., Malakuti, S., Grüner, S., Goldschmidt, T., 2019. Interoperable digital twins in IIoT systems by transformation of information models: a case study with asset administration shell. In: 9th International Conference on the Internet of Things. ACM, Bilbao Spain, pp. 1–8. <https://doi.org/10.1145/3365871.3365873>
- Platenius-Mohr, M., Malakuti, S., Grüner, S., Schmitt, J., Goldschmidt, T., 2020. File- and API-based interoperability of digital twins by model transformation: an IIoT case study using asset administration shell. *Fut. Gen. Comput. Syst.* 113, 94–105. <https://doi.org/10.1016/j.future.2020.07.004>
- Plattform Industrie 4.0, 2025. Plattform Industrie 4.0 - Homepage. Accessed: 2025-02-24. <https://www.plattform-i40.de/IP/Navigation/EN/Home/home.html>
- Systems Modeling Community, 2025. Vehicle definitions example. Accessed: 2025-03-02. <https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation/blob/master/sysml/src/examples/Vehicle%20Example/VehicleDefinitions.sysml>
- Schmidt, C., Volz, F., Stojanovic, L., Sutschet, G., 2023. Increasing interoperability between digital twin standards and specifications: transformation of DTDL to AAS. *Sensors* 23 (18), 7742. <https://doi.org/10.3390/s23187742>
- Schmidt, D.C., 2006. Guest editor's introduction: model-driven engineering. *Computer* 39 (02), 25–31.
- Troya, J., Segura, S., Burgueño, L., Wimmer, M., 2023. Model transformation testing and debugging: a survey. *ACM Comput. Surv.* 55 (4), 1–39. <https://doi.org/10.1145/3523056>
- Ulitin, B., Babkin, E., Babkina, T., Vizgunov, A., 2019. Automated formal verification of model transformations using the invariants mechanism. In: *Perspectives in Business Informatics Research*. Springer, 365, pp. 59–73.
- Wolny, S., Mazak, A., Carpella, C., Geist, V., Wimmer, M., 2020. Thirteen years of sysML: a systematic mapping study. *Softw. Syst. Model.* 19 (1), 111–169. <https://doi.org/10.1007/s10270-019-00735-y>
- Zhang, J., Ellwein, C., Heithoff, M., Michael, J., Wortmann, A., 2025. Digital twin and the asset administration shell: an analysis of the three types of AASs and their feasibility for digital twin engineering. *Softw. Syst. Model.* 24 (3), 771–793. <https://doi.org/10.1007/s10270-024-01255-0>